Roll Number

0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9

The OMR Sheet will be computer checked. Fill the circles completely and dark enough for proper detection. Use Dark HB pencil or ballpen (black or blue) for marking. While erasing, erase properly.

Avoid Improper Marking

Partially Filled

Lightly filled

Analysis & Design of Algorithms 2026 — Quiz I

Duration: 1 hour Total Marks: 30

Name: _____ **Roll No.:** _____

Problem 1. (15 points) You are given a set $S = \{(x_1, w_1), (x_2, w_2), \ldots, (x_n, w_n)\}$, where each $x_i \in \mathbb{R}$ is a key and each $w_i > 0$ is a positive weight. Let $W = \sum_{i=1}^{n} w_i$ denote the total weight. An element x_k is called a *weighted median* of S if it satisfies

$$\sum_{x_i < x_k} w_i \leq \frac{W}{2} \quad \text{and} \quad \sum_{x_i > x_k} w_i \leq \frac{W}{2}.$$

Design a deterministic algorithm that finds a weighted median of S in $O(n)$ time.

(a) (10 points) Write the pseudocode with inputs, output and arguments to all functions.

The only crucial thing you need to realize is that the weights do not change anything significant other than how your recursive calls will look like. More precisely, the choice of pivot can still be the Median-of-Medians OR you may even call the median selection algorithm to select the (unweighted) median of the whole array as the pivot.

Algorithm 1 WeightedMedian(S, T_L, T_R)

Input: A set S of pairs $\{x_i, w_i\}_{i=1}^{n}$ and two target weights T_L for the left of the median, T_R for the right of the median (initial call uses $W/2$ for both).
Output: The weighted median key x_k. {+1 for any correct base case}
if $|S| = 1$ **then**
  return the key $x_i \in S$
end if
$m \leftarrow$ DeterministicSelect$(S, \lceil |S| \rceil / 2)$ {+2 for finding pivot correctly using $O(n)$ algorithm}
Use Partition Algorithm to find the following sets : {+1 for this}
$L \leftarrow \{(x_i, w_i) \in S \mid x_i < m\}$
$R \leftarrow \{(x_i, w_i) \in S \mid x_i > m\}$
$W_L \leftarrow \sum_{(x_i, w_i) \in L} w_i$
$W_R \leftarrow \sum_{(x_i, w_i) \in R} w_i$
{+2 for **each of the follwing three** cases: +1 for the correct condition check and +1 for the correct recursive call/return}
if $W_L \leq T_L$ **and** $W_R \leq T_R$ **then**
  return m
else if $W_L > T_L$ **then**
  return WeightedMedian$(L, T_L, T_R - W_R - w_m)$
else
  return WeightedMedian$(R, T_L - W_L - w_m, T_R)$
end if

Remarks :

1. Some of you might be using the Median-of-Medians directly as pivot. This is also absolutely fine but then your runtime analysis should reflect that
2. Some of you might have designed an algorithm using *only one parameter for both the left weight and right weight.* They will receive 8/10 for the pseudocode *provided everything else is fine and the one parameter value reflects the difference in weight that you need to cover*

(b) (5 points) Justify the linear runtime. You may use results from lectures without proofs.

- **Pivot Selection:** Finding the median m of the keys using a deterministic selection algorithm takes $O(n)$ time (In case of median-of-medians, you will have to say $O(n) + T(n/5)$ or something similar)

- **Partitioning:** Creating the sets L, R and calculating W_L requires $O(n)$ time.
- **Recurrence Relation:** Since m is the median of the keys, the algorithm recurses on a set of size at most $n/2$ The recurrence is $T(n) = T(n/2) + O(n)$. In case pivot is median-of-medians, the recurrence should be $T(n) \leq T(n/5) + T(7n/10) + O(n)$
- **Conclusion:** By the Master Theorem (or in the second case, 'from lectures'), the geometric reduction of work results in a total runtime of $O(n)$

Rubric : +1 for each of the first, second and fourth bullet. +2 for the third.

Problem 2. (15 points) Given an array $A[1..n]$ of integers (which may include negative values), design a $\Theta(n \log n)$-time *divide and conquer style* algorithm to find indices $1 \leq i \leq j \leq n$ such that the sum $\sum_{k=i}^{j} A[k]$ is maximized. The output subarray must be *non-empty.*

(a) (10 points) Write pseudocode. Mention input, output and all function arguments clearly.

Brief Explanation (Not required from you) : The idea is to divide the array in the middle. Then recursively compute the max sum subarrays of the two halves, The combine step is to compute the max-sum subarray *which includes the middle element.* It is easy to observe that this can only be the concatenation of the maximum sum *suffix* of the left half and the maximum sum *prefix* of the right half. We first write a subroutine MaxCrossingSubarray for this step and then the main Divide and Conquer algorithm. There exists an $O(n)$ algorithm to solve this problem. But that is not divide and conquer, neither is it $\Theta(n \log n)$ and hence you will get a **zero** for writing that.

Algorithm 2 MaxCrossingSubarray$(A, low, mid, high)$

Input: Array A, indices $low, mid, high$.
Output: A tuple $(max\_left, max\_right, sum)$.
$left\_sum \leftarrow -\infty$, current_sum $\leftarrow 0, max\_left \leftarrow mid$
for $i = mid$ **downto** low **do**
  $current\_sum \leftarrow current\_sum + A[i]$
  if $current\_sum > left\_sum$ **then**
    $left\_sum \leftarrow current\_sum$
    $max\_left \leftarrow i$
  end if
end for
$right\_sum \leftarrow -\infty$, current_sum $\leftarrow 0, max\_right \leftarrow mid + 1$
for $j = mid + 1$ **to** $high$ **do**
  $current\_sum \leftarrow current\_sum + A[j]$
  if $current\_sum > right\_sum$ **then**
    $right\_sum \leftarrow current\_sum$
    $max\_right \leftarrow j$
  end if
end for
return $(max\_left, max\_right, left\_sum + right\_sum)$

Rubric: +2.5 for implementing each of the sums.

(b) (5 points) Justify the runtime.

- **Divide:** Calculating the midpoint takes $\Theta(1)$ time.
- **Conquer:** The algorithm recursively solves two subproblems, each of size $n/2$. This contributes $2T(n/2)$ to the recurrence.
- **Combine:** The `MaxCrossingSubarray` function performs two linear loops (one from mid to low and one from $mid+1$ to $high$), which takes $\Theta(n)$ time.($\Theta(n)$ is crucial here, you get only +0.5 for writing $O(n)$)
- **Recurrence:** The total time $T(n)$ is governed by:

$$T(n) = 2T(n/2) + \Theta(n) \tag{1}$$

Algorithm 3 MaxSumSubarray$(A, low, high)$

Input: Array A, range indices $low, high$.
Output: A tuple (i, j, sum) representing the max subarray in $A[low..high]$.
if $low = high$ **then**
  return $(low, high, A[low])$ {+1 for base case}
end if
$mid \leftarrow \lfloor (low + high)/2 \rfloor$ {+1 for each of the following calls provided mid is computed correctly above}
$(left\_i, left\_j, left\_sum) \leftarrow$ MaxSumSubarray(A, low, mid)
$(right\_i, right\_j, right\_sum) \leftarrow$ MaxSumSubarray$(A, mid + 1, high)$
$(cross\_i, cross\_j, cross\_sum) \leftarrow$ MaxCrossingSubarray$(A, low, mid, high)$

return the tuple with the largest sum among the three results.{+1 for this}

- **Conclusion:** You can directly apply Master Theorem Case 1 since it actually gives you Θ. Otherwise, any recurrence tree based short argument is fine too.

Rubric : +1 for each of the above.